

EdSharp User Guide

EdSharp\ Version 4.0\ May 29, 2017\ Copyright 2007 - 2017 by Jamal Mazrui\
GNU Lesser General Public License (LGPL)\

Contents

- [Description](#)
- [Installation](#)
- [Editing](#)
- [Navigating](#)
- [Querying](#)
- [Managing Files](#)
- [Invoking Snippets](#)
- [Working with Structured Text](#)
- [Word Processing](#)
- [Doing Math](#)
- [Programming](#)
- [Scripting Add-Ins](#)
- [Miscellaneous](#)
- [Hotkey Summary](#)
- [Development Notes](#)

Description

EdSharp is a full featured text editor that is friendly, powerful, and open source. It uses a standard Windows interface for an application that supports multiple document windows. Though intended for sighted users as well, it seeks to enhance productivity for users of the JAWS, NVDA, Window-Eyes, or System Access screen readers by automatically verbalizing relevant information. These speech messages supplement default speech heuristics, providing confirmation or results of commands without the need for manually interrogating the screen. If a screen reader is not detected in memory, EdSharp uses the default SAPI voice, if available, which may be configured via the Speech applet in Control Panel.

Written in the C# (pronounced C Sharp) language, EdSharp implements the "Homer editor interface," which originally evolved with an editor called TextPal. The same interface was also implemented in the package of JAWS scripts and tools called HomerKit. EdSharp requires the .NET Framework 4.0 or above to run: a free download from Microsoft that is also installed with Windows 7 or later.

Almost every EdSharp command may be done through a mnemonic keystroke, as well as a menu or mouse operation. These commands begin with the standard keys available in Notepad or most Windows-based editors. EdSharp then adds many beneficial features. Optional scripts for some screen readers provide further fine tuning of the speech interface for those users.

Installation

The installation program for EdSharp is called EdSharp_setup.exe. When executed, it prompts for a program folder, the default being `C:\Program Files (x86)\EdSharp`. The installer also creates a program group for EdSharp on the Windows start menu, containing choices to launch EdSharp, read Documentation, or uninstall. Additional choices either set or clear an association between EdSharp and files with a particular extension, such as .txt or .ini. Binary formats such as .pdf or .pptx may also be associated with EdSharp, thus permitting automatic conversion to text when opened via Windows Explorer.

After installing EdSharp, the setup program presents a list of several checkboxes. One checkbox offers an optional set of JAWS scripts to fine tune the EdSharp speech interface in a few ways that could not be accomplished otherwise. If the scripts were installed and you would later prefer default JAWS behavior instead, you can disable the scripts via the "Manage Application Settings" dialog from the "Options menu of JAWS.

Another checkbox sets Alt+Control+E as a system-wide key associated with the EdSharp shortcut placed on the Windows desktop. If this hot key is found to conflict with an existing shortcut, navigate to either the EdSharp or other shortcut on the desktop, press Alt+Enter for properties, and then change the hot key to something else (or leave it blank).

An additional checkbox opens this manual in the default web browser.

EdSharp may be safely installed over previous versions. The About option from its Help menu, or Alt+F1 key, indicates the current version number and release date. The History of Changes command, Shift+F1, summarizes fixes and improvements over time.

EdSharp features may be explained in the following categories of activity: editing, navigating, querying, managing files, invoking snippets, working with structured text, word processing, programming, doing math, and miscellaneous.

Editing

Selecting, Copying, and Pasting

As usual, you may press Control+C or Control+X to copy or cut selected text to the clipboard. EdSharp tries to make an intelligent guess when no text is selected. In this case, the current line is assumed. Press Alt+C or Alt+X to perform a copy or cut operation that appends rather than replaces text on the clipboard. If the previous clipboard text did not end with a line break, EdSharp inserts one before the appended text.

An alternative way of selecting text uses F8 to mark the start of a selection. Navigate to the end point by whatever means (arrow keys, find command, etc.) without having to hold down the Shift key. Note that the caret should be placed one position past the last character to be selected. Press Shift+F8 to select text from the start position. EdSharp says the number of characters selected. You may subsequently select between the same positions again with Control+Shift+F8, or return to the start position with Alt+Shift+F8.

As usual, Control+A selects all text. Control+Shift+A clears any selection. Press Control+F8 to copy all text to the clipboard with a single command. Control+V pastes clipboard text at the cursor position. Control+Shift+V inserts the content of a file instead.

Press Control+Space to select a chunk of text at the cursor position. A chunk is defined as a contiguous sequence of non-space characters. It may be more than what the hotkey Control+Shift+RightArrow selects, since such word movement commands stop at punctuation marks. Press Control+Space again to extend the selection to the next chunk.

The Append From Clipboard command, Alt+7 (associate append with the ampersand character), is like the "paste board" feature of the NoteTab editor. When this mode is toggled on for a document, each snippet of text copied to the Windows clipboard from any application will also be pasted into the document, which will then automatically be saved to disk. EdSharp beeps to confirm this is happening. Snippets are separated by a section break sequence,

which makes it possible to navigate among them with the Control+PageDown and Control+PageUp commands. Thus the feature may be used to conveniently collect and save information from applications that do not have a built-in appending mechanism. When done, toggle off the mode with the same command, Alt+7.

Press Control+Z to undo the last editing operation, or Control+Shift+Z to redo it.

Press Control+N to start a new document, or Control+Shift+N to initialize one with text on the clipboard.

Replacing

Press Control+R to replace text and hear the number of matches. Press Control+Shift+R to replace with a "regular expression" -- a complex but powerful syntax that permits almost any transformation of text. EdSharp replaces within selected text, or all text if there is no selection. Press Control+Shift+E to extract parts of text based on a regular expression. The matching parts are placed in a new editing window separated by a section break sequence of characters. EdSharp uses regular expressions of the .NET Framework, explained at [http://msdn2.microsoft.com/en-us/library/hs600312\(vs.80\).aspx](http://msdn2.microsoft.com/en-us/library/hs600312(vs.80).aspx)

Press Alt+Shift+P to copy the full path of the current file to the clipboard, permitting it to be easily pasted into dialogs of other applications. Use the Path List command, Control+Shift+P, to generate a list of files in a new editing window. You are prompted for the directory to open, and then the extensions to include based on what files EdSharp finds in the directory. The new window contains the full path of the first file, and then just file names on subsequent lines -- since they are in the same directory.

Changing Case

Press Control+U to convert the current character or selected text to upper case, or Control+Shift+U to lower case. Alt+U converts to proper case, putting the first letter of each word in upper case and the rest in lower case. Alt+Shift+U

inverts case, converting lower case characters to upper case and vice versa.

The Yield Encoding command, Alt+Shift+Y, may be used to convert all or selected text according to a particular character encoding. If text from a file or the clipboard appears to be rendered improperly in EdSharp, you can tell it to base its interpretation on a different encoding: ASCII, Latin 1, UTF-7, UTF-8, UTF-16, UTF-32, or another encoding that you pick from a list of over 100 available. You can also choose a conversion where the Unicode number of each character is put on a separate line. This may be used to identify non-printing characters in the document. The command replaces all or selected text, so put a copy in a new document window if you want to retain the original.

Use the Quote command, Control+Q to add an email quote sequence (>) at the start of the current or selected lines. Control+Shift+Q removes this sequence, as well as any other leading space or tab characters.

F2 prompts for a character to insert based on its numeric value in the Unicode character set. The number should have four hex digits (base 16). This command is useful for inputting a character that does not have a corresponding keystroke. You can specify a decimal (base 10) number instead by preceding it with the letter d. For example, the ellipses symbol (...) may be specified either in hex as 2026 or in decimal as d8230.

Press Control+Shift+J to join lines of all or selected text. A sequence of any number of spaces followed by a hard line break is replaced by a single space. However, consecutive line breaks denoting a paragraph break are not effected. Generally, the purpose is to word wrap paragraphs that contain unnecessary line breaks, e.g., text received in an email message.

The reverse command is Hard Line Break, Control+Shift+H. It lets you specify the maximum width of lines in all or selected text. EdSharp prompts for the number of characters allowed before a line break, defaulting to the number currently found as the width of text. Generally, the purpose is to format text for a display that does not automatically word wrap.

Comparing and Sorting

Several commands help you compare and sort textual items. The LimitItem

setting in the Configuration Options dialog specifies the divider between items, which is a hard line break by default (`\n`) -- lines are unwrapped for this purpose. Use the Trim Blanks command, `Control+Shift+Enter`, to eliminate space or tab characters at the start and end of a line in the current line or selected text. Consecutive blank lines are also reduced to a maximum of two.

Use the Order Items command, `Alt+Shift+O`, to alphabetically sort lines in all or selected text. Press `Alt+Shift+Z` to reverse the order of all or selected lines (you may think of a reverse order from Z to A). Use the Keep Unique Items command, `Alt+Shift+K`, to eliminate duplicate lines from all or selected text. EdSharp ignores case when comparing lines in these commands.

Press `Alt+Shift+N` to number lines in all or selected text. EdSharp prompts for the starting number, defaulting to 1. Each line is then prefixed by a consecutive number, period, and space. Blank lines are ignored.

Two commands produce text in a new window after comparing sets of lines. Lines above the cursor are considered the first set, and the rest are considered the second. Use the List Different Items command, `Alt+Shift+L`, to get lines that are in the first set but not the second. Use the Query Common Items command, `Alt+Shift+Q`, to get lines that are in both. These commands ignore blank lines but are case sensitive.

Deleting

As usual, Delete (without a selection) deletes a character in the forward direction, and Backspace deletes backward. `Control+Delete` deletes forward by a word, and `Control+Backspace` deletes backward. `Control+Shift+Delete` deletes from the cursor to the end of the line, and `Control+Shift+Backspace` deletes from the cursor to the start of the line. `Alt+Shift+Delete` deletes from the cursor to the end of the document, and `Alt+Shift+Backspace` deletes from the cursor to the top of the document. `Alt+Backspace` deletes the current line. `Control+D` deletes the current hard line (past wrapping to the next hard line break). `Control+Shift+D` deletes the current paragraph (past one or more blank lines). After deleting, EdSharp reads the new character, word, or line at the cursor.

Press F7 to spell check all or selected text. Use the Thesaurus command, Shift+F7, to look up synonyms for the word at the cursor position. These features rely on an installation of Microsoft Word (and use the same hot keys).

Navigating

Press Home or End to go to the start or end of the line, and automatically hear the character there. Press Alt+Home or Alt+End to go to the first or last non-blank character of the line. Press Control+Home or Control+End to go to the top or bottom of the document, and automatically hear the line there.

As usual, Control+F finds text in a forward direction. Control+Shift+F reverses the search. Alt+F3 or Alt+Shift+F3 search forward or backward for either the chunk at the cursor or selected text. Control+F3 or Control+Shift+F3 prompt for a regular expression for searching forward or backward. F3 or Shift+F3 searches forward or backward for the last target, which may be either standard text or a regular expression. If a search is successful, EdSharp reads the matching line automatically.

The Text you enter in Find or Replace dialogs may include tokens that represent nonprinting characters. This syntax is available for strings in the C programming language and its variations. Common tokens are a pair of characters consisting of a backslash and letter, such as the following: \r for carriage return (ASCII 13), \n for line feed (ASCII 10), \t for tab (ASCII 9), and \f for form feed (ASCII 12). Such tokens allow you to search for text, say, at the beginning or end of a line (use \n for a line break in EdSharp).

The trade off for this flexibility is that backslash and quote characters must be preceded by a backslash when intended literally (not part of a token), i.e., \ for backslash and \" for quote. Since this doubling of characters may be cumbersome with search terms such as a file path, however, EdSharp supports use of an initial @ character to indicate that the following characters should be interpreted literally rather than as possible tokens. For example, if searching for a file in the document, you could enter the term \\ @c:\\temp\\temp.txt rather than \\ c:\\temp\\temp.txt. If you need to search for a string that begins with the @ character, precede it with a backslash, e.g., \\ @domain.com follows the user name of an email address.

Press Control+G to go to a percentage point in the document, or Alt+G to repeat the command with the previous value. Similarly, press Control+J to

jump to a line number, or Alt+J to repeat. A column number may be specified after the line number and a comma. If no line number is specified before the comma, EdSharp jumps to the column on the current line.

Press Control+RightArrow or Control+LeftArrow to read by word -- stopping at embedded symbols. Press Alt+RightArrow or Alt+LeftArrow to read by chunk -- text delimited only by white space characters. Press Alt+DownArrow or Alt+UpArrow to read by sentence. Press Control+DownArrow or Control+UpArrow to read by paragraph.

Querying

Use the Address command, Alt+A, to hear the line, column, and percent position of the cursor in the document. Press Alt+P to hear the full path of the file on disk. Use the Yield command, Alt+Y, to hear the number of characters, words, and lines contained in all or selected text. Press Alt+Z to hear whether the document has been modified from the version on disk, or press it again to check its character encoding.

Press Alt+F8 to hear the whole document without moving the cursor. Use the Quote Clipboard command, Alt+Apostrophe, to hear the textual content of the clipboard, or its spelling if this key is pressed again. Press Alt+Semicolon to hear the current time and date.

Press Shift+Space to hear selected text. Press Shift+Backspace to hear the chunk of text at the cursor. Press Shift+F4 to hear the titles of open document windows. If the same key is pressed again without moving the cursor, the text is spelled instead.

Press Control+Shift+Y for the "yield," or number of results matching a regular expression, which you specify. This may be useful before using Control+Shift+R to replace text or Control+Shift+E to extract it.

Managing Files

As usual, press Control+O to open a file. Press Control+Shift+O to open a file in a format other than plain text. A converter is invoked based on the file extension.

File converters may be configured through the Manual Options command, Alt+Shift+M. For example, an open source PDF converter is distributed with EdSharp and configured, by default, with the following line in the Import section:
`\pdf=%ProgDir%\pdftotext.exe %Source% %Target%`

To configure a converter, specify the command line for converting an extension from a source, non-text format to a target, text format. The following variables may be used:\

- %ProgDir% = Full path of the directory containing the EdSharp.exe program
- %Source% = Full path of the source file
- %SourceDir% = Full path of the directory containing the source file
- %SourceName% = Name of the source file
- %SourceRoot% = Root name without extension of the source file
- %SourceExt% = Extension of the source file

The short path of a file or directory is used unless a variable includes a Long suffix, e.g., %ProgDirLong% or %SourceLong%. Most utilities require long file names to be surrounded by quote marks, e.g., "%SourceLong%" syntax. For technical reasons, if quotes are used within the command line, then a pair of quotes should also be added around it. Variables for the Target file are like that of the source.

External converters distributed with EdSharp are stored in the Convert subfolder of the EdSharp program folder, e.g., in (default installation)\
C:\Program Files (x86)\EdSharp\Convert

A text format called Markdown is useful for various conversions, explained at <http://en.wikipedia.org/wiki/Markdown>

If EdSharp finds more than one converter available for a file extension, you are prompted which one to use. If a converter entry does not contain the digit 2 and another extension, it is assumed to be .txt.

Use the Open Again command, Alt+O, to reload the current file from disk. Press Alt+R to open a file from the list of those recently used. Use the FileFind command, Alt+Shift+F, to pick a file from a list of those containing text and matching wildcards that you specify. Multiple wildcard patterns are possible, separated by a vertical bar (|) character, e.g., *.txt|catalog*.htm

Handling Favorite Files and Bookmark Positions

Press Control+L to add the current file to the list of favorites. Press Control+Shift+L to remove it from that list. Press Alt+L to list favorites and open one. If a bookmark is set, EdSharp automatically goes to it and says the percent position in the document. Also, if the word wrap or guard setting is different than when the file was designated as a favorite, EdSharp restores that setting and says so. This command opens a file verbatim, assuming that you had set it as a favorite to edit it literally, e.g., a .htm file you are developing. When a file is opened from Windows Explorer or the recent files list, on the other hand, EdSharp automatically converts it to plain text if an import converter is configured for its extension.

Press Control+K to set a bookmark at the cursor position. If the current file has a single bookmark, Alt+K goes to it. If more than one, a list of bookmarked lines is presented, with focus on the next one ahead of the cursor position. Thus you can sequentially visit bookmarks by pressing Alt+K and Enter.

Control+Shift+K clears a bookmark at the cursor position. To clear all bookmarks at once, press Control+Shift+L to remove the file from the list of favorites -- since a bookmarked file is automatically considered a favorite. If you want to keep the favorite status without any bookmarks, then press Control+L to set the file as a favorite again. EdSharp tracks and restores the bookmark, word wrap, and guard settings of each file opened.

Saving

As usual, press Control+S to save. Press Control+Shift+S to Save As, giving

the document a new name in EdSharp and on disk. Press Alt+Shift+S to save a copy of the document under a different name while keeping the original name in EdSharp.

When saving text to a file, EdSharp checks whether any character has a Unicode number greater than 255, which means that more than one byte is needed to represent it. If so, the file is saved with a UTF-8 encoding, the most common form of Unicode for storing files on disk. Otherwise, the default encoding of the computer is used, e.g., Latin 1.

Alt+Shift+E exports a file to another format. Built-in options include ASCII format (characters with ANSI codes above 127 are removed), Mac format (line break is \r), and Unix format (line break is \n). Via Microsoft Word converters, additional formats include .doc, .htm, .rtf, and .xml. Other converters may be configured by editing the Export section through the Manual Options command, Alt+Shift+M. The syntax is like that in the Import section (explained elsewhere). The Other option lets you pick a character encoding for the target file from a list of over 100 available.

Use the Run command, F5, to execute the current file as if its name had been entered in the Windows Start/Run dialog. The effect is also like pressing Enter on its name in Windows Explorer, opening it with the program associated with that extension. For example, pressing F5 when the current document has a .htm extension will open it in the default web browser.

Press Shift+F5 to execute a file path, email address, or web URL at the cursor position. If text is selected, it will be used instead -- after removing any line break characters. You also have a chance to adjust it before execution.

Use the Mail command, Control+M, to send the current file as the body of a message, or Control+Shift+M to send it as an attachment. These commands invoke a Windows feature similar to the "Send To" feature of Microsoft Word.

Press Alt+Shift+R to rename the current file, both in EdSharp and on disk.
Press Alt+Shift+D to delete it.

Press Alt+Backslash to open Windows Explorer in the directory containing the current file, or Control+Backslash to open a command prompt there. Besides

the current folder as the default to open, the intervening dialog also lets you open the EdSharp program folder, data folder, or snippet folder. It also lets you create a new folder on disk.

Press Alt+0 to verify the current compiler and folder of EdSharp. Control+0 lets you change the current folder to one containing recent or favorite files, which are put in a list. Press Control+Shift+0 to change to a special folder of Windows, e.g., My Documents. These commands may be more efficient than navigating the standard Windows open file dialog, invoked with Control+O.

Invoking Snippets

Press Alt+S to save all or selected text to a file that may be conveniently pasted into other documents. You may give the file a descriptive name and an extension appropriate for its content. It is saved in a subfolder of the EdSharp data folder. Each programming compiler or interpreter may have its own set of snippets. The subfolder name is the same as the current value of the Pick Compiler command, Control+Shift+F5. If no compiler has been chosen, the "Default" subfolder is used.

Press Alt+V to pick one of the available snippets and paste it into the current document. This command lists snippets in the Default snippet folder, as well as those in the folder associated with a Compiler being used. This lets you have a set of snippets that are available regardless of the programming language in use.

Use the View Snippet command, Alt+Shift+V, to load a snippet file into EdSharp for viewing or editing rather than execution. You can also manage snippet files with the Explorer Folder command, Alt+Backslash, which lets you open Windows Explorer in the subfolder containing snippet files.

EdSharp processes a snippet with a .js extension as JScript .NET code to be evaluated. Such a file can do almost anything in EdSharp, as explained in the section about EdSharp's scripting capability. For example, the "ul from selected.js" file generates an HTML unordered list from selected lines of text.

A Non .js snippet may be either literal or an interpreted type. A literal snippet is pasted completely. An interpreted type is separated into an initial header line and remaining lines as its body. The header line contains keywords, in lower case, that control how EdSharp processes the snippet. At present, two interpreted types are defined: html and text. The type keyword must be the first word on the header line, and thus the first word of the snippet file.

The first body line of an HTML snippet is the name of an HTML tag. Subsequent body lines are attributes of the tag. An optional default value can follow the attribute name, separated by an equals sign (=). Here is an example

for the anchor tag:

html phrase a href= name= target= class= title= src=

An attribute may contain \n or other nonprinting tokens. It may be commented out with a semicolon (;) as the first character of the line.

The "phrase" keyword tells EdSharp that the tag may be embedded in a paragraph, rather than creating a block with line breaks before and after. Another keyword, "empty," would tell EdSharp not to add a closing tag like when pasting (e.g., for the tag).

EdSharp pastes only those attributes that have values greater than zero in length as part of the opening tag. To include an attribute with essentially no value, enter a space character for it in the dialog. If text is selected when pressing Alt+V, it is surrounded by the opening and closing tags, and the cursor is placed afterward. If there is no selection, the cursor is placed between the opening and closing tags.

Over 100 HTML snippets are distributed with EdSharp: a collection of tags and attributes to serve common needs in developing web sites. HTML and PHP page templates are also included. You can modify or add to these, and are encouraged to submit ones you think would be useful to others.

New .txt and .js snippet files will be installed when upgrading EdSharp. Since the installer does not replace snippets with the same names, however, you need to manually clear the appropriate folder if you want to ensure a fresh set of snippets. You can do this by pressing Control+Shift+F5, picking the HTML Tidy compiler, then pressing Alt+Backslash and choosing the snippet folder to open in Windows Explorer. From that window, press Control+A and Delete to remove all files in the folder.

With the text type of snippet, EdSharp pastes the whole body after making possible substitutions controlled by a keyword called "form." This lets you embed variable or constant tokens in the body. A variable has surrounding percent signs and an equals sign between the name and default value. For example, the variable %City=%Silver Spring% means a variable named City

with a default value of Silver Spring. EdSharp creates a dialog that prompts for the value of each variable it finds in the snippet body. It then replaces the variable references with the values entered. You may repeat the same variable reference in a snippet so the user is prompted once for a value that is then used for multiple text insertions. Subsequent references should omit the default value after the equals sign, e.g., a %City=% token.

Certain constant tokens are also defined: %Date% for the current date, %Time% for the time, %UserName% for the Windows user name, %UserFirstName% for the first part of that name, and %UserLastName% for the second part, if any. Date and Time formats may be customized as EdSharp configuration options, using the DateTime formatting syntax of the .NET Framework, explained at [http://msdn2.microsoft.com/en-us/library/system.globalization.datetimeformatinfo\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/system.globalization.datetimeformatinfo(VS.80).aspx)

If a snippet header contains the "caret" keyword, EdSharp looks for a double caret sequence (^) in the body, and positions the cursor (think of blinking caret) in that location after pasting. An example text type snippet is called Letter.txt, located in the Default snippet folder. Its content is as follows:

```
``` text form caret %Date%
```

```
Dear %Customer=%:
```

```
Thank you for your purchase of %Product=Super Widget%. ^^
```

```
Sincerely, %UserName% ```
```

EdSharp notices that this is a text type snippet because of the first word of the file. It finds two other keywords on the header line: form and caret. It creates a dialog with two edit boxes, prompting for the Customer and Product -- defaulting to Super Widget. It substitutes the values entered, as well as date and user name constants. After pasting, the cursor is positioned after the first sentence.

A section of the configuration file that supports snippets is called Tokens. Each of these user-defined tokens is an expression in Microsoft JScript .NET: a version of JavaScript explained at <http://msdn2.microsoft.com/en->

[us/library/t895bwkh\(VS.80\).aspx](us/library/t895bwkh(VS.80).aspx)

Three token examples are currently provided in the EdSharp configuration file. The CurrentDirectory token illustrates a call to a static method in the .NET Framework Class Library (FCL) -- in this case, returning the current directory of the EdSharp process. The Signature token shows syntax for a literal string -- in this case, a signature block with multiple lines. The UnorderedList token refers to a JScript file called ul.js that is provided in the HTML snippet folder.

When EdSharp finds that a token refers to a file in the current snippet folder, it interprets the content of that file as JScript. The ul.js example creates an unordered list element in HTML after prompting for the number of items to generate in the list. Its content is as follows:

```
[Begin Content of ul.js] var iCount =
Interaction.InputBox("Number of Items:", "Input", "0") var
sTag = "\n" var i = 1 while (i <= iCount) { sTag += "
Item" + i + "\n" i++ } sTag += "\n" [End Content
of ul.js]
```

User-defined tokens may be included in a snippet that has the "form" keyword in its header. They may also be typed in a document being edited. A token expression may be tested with the Evaluate Expression command, Control+Equals, which evaluates the current line or selected text and places the result on the line below. The Replace Tokens command, Control+Shift+Equals, swaps token names with their computed results in all or selected text.

For example, you might press Alt+Shift+M for Manual Options and define a signature token as follows:

```
[Tokens] Signature=("Sincerely,\nJohn
Doe\nJohn.Doe@NiftyHomePage.com\n")
```

Then type %Signature% in your document where you want that to appear, and use the Replace Tokens command to do it. Alternatively, put %Signature% in the body of a snippet containing the form keyword in its header, and paste that snippet with Alt+V.

## Working with Structured Text

Several commands work with a structured text document consisting of a table of contents that lists topics at the beginning of the document, followed by a section for each topic in the body. Each topic in the table of contents is a line with the same text as the heading of its corresponding section. A divider sequence of characters -- a line of 10 dashes followed by a form feed and line break, separates each section. The next line after such a section break is the topic heading of the next section. For an example of this structure, examine the EdSharp.txt file in the EdSharp program folder. The default location is\

```
C:\Program Files (x86)\EdSharp\EdSharp.txt
```

Press Control+PageDown to go to the next section and read its heading, or Control+PageUp to go to the previous one. When the cursor is on a topic in the table of contents, press F6 to go to its corresponding section in the body. Press Shift+F6 to go from a section in the body to its topic in the table of contents. Press Control+F6 to search for a topic based on text within its heading. The search starts at the beginning of the document. Press Alt+F6 to search again for the next match.

To create a structured text document, press Control+Enter to insert a section break at the cursor position, then type the heading of the next section. Alt+T verbally confirms the topic of the current section. Use the Text Contents command, Alt+Shift+T, to generate and prepend a table of contents to the current document. The first line of the document becomes the first topic in the table of contents. Each line of text after a section break becomes another topic.

You can adjust the LimitItem configuration setting to perform comparison operations on sections rather than lines of text. For example, press Alt+Shift+C for Configuration Options, and Alt+S for the SectionBreak setting. Since the text is initially selected, press Control+C to copy it to the clipboard. Then press Alt+L for LimitItem, Control+V to paste, and Enter to save settings. Now you can sort sections alphabetically with the Order Items command (Alt+Shift+O), reverse them with the Reverse Items command (Alt+Shift+Z), or eliminate duplicates with the Keep Unique Items command (Alt+Shift+U).

# Word Processing

EdSharp supports several aspects of Rich Text Format (.rtf) as well as plain text (with optional structure). In certain situations, EdSharp behaves differently if a file has a .rtf extension rather than any other one. Specifically, the Open Other Format command, Control+Shift+O, imports a .rtf file with its formatting rather than converting it to plain text. The Save, Save As, and Save Copy commands, Control+S, Control+Shift+S, and Alt+S, save a .rtf file with formatting preserved. The Print command, Control+P, prints a .rtf file using the associated program for this operation in the Windows registry (typically Microsoft Word or WordPad). Use the Copy Rich Text command, Control+Shift+C, to copy selected text with formatting to the clipboard.

Formatting commands include the following. Use the Justify command, Alt+Shift+J, to set the horizontal alignment of text as left, bullet, center, or right. This formatting applies to either selected text or the current hard line -- a line of text terminated by a hard line break (created by pressing Enter rather than wrapping).

Use the Style command, Alt+Slash, to set or clear bold, italic, or underline formatting. This applies to either selected text or text ahead of the current cursor position. The BaseLine command, Alt+Shift+6 (think of a caret), creates a subscript or superscript with selected text or text ahead of the cursor. The Selection Font command, Alt+Shift+Dash, adjusts the font or color of text (think of a "dashing" display). The key to its right, Alt+Shift+Equals, is for setting the default font of a new document. These Justify, Style, Baseline, and Font dialogs also indicate current format settings.

Navigation commands let you move forward or backward to a change in formatting. Control+RightBracket goes to the next justification change, and Control+LeftBracket goes to the previous one. Control+6 goes to the next baseline change, and Control+Shift+6 goes to the previous one. Control+Slash goes to the next style change, and Control+Shift+Slash goes to the previous one. Control+Dash goes to the next font change and Control+Shift+Dash goes to the previous one. The cursor stops at the character with different formatting. The new formatting is announced and current context is read.

To query the current font and color, press Alt+Dash. For styles, baseline, and justification, press Alt+Slash.

## Doing Math

You can press Control+Equals to evaluate mathematical expressions in JScript code, either on the current line or in selected text. Control+Shift+G goes to another, more interactive environment, such as the interactive console of Python or iPython. The latter is particularly useful for learning the library of the .NET Framework and testing expressions that may be incorporated in programming code, including EdSharp snippets. It may also be used as a simple, speech-friendly calculator.

Alternatively, the GoToEnvironment setting could be configured for a computer algebra system such as Maxima, \ <http://Maxima.SourceForge.net> or Axiom, \ <http://Axiom.SourceForge.net>

## LaTeX

EdSharp includes support for the LaTeX language (pronounced La Tech). This is a common language used for typesetting, especially for scientific publications. About 30 sample LaTeX snippets, ending in a .tex extension, are distributed with EdSharp. You can convert from LaTeX to RTF and vice versa. To fully work with LaTeX, install the open source package for Windows from \ <http://www.miktex.org>

With that installation, EdSharp's LaTeX compiler option lets you check and correct syntax. You can then export to PDF or XML -- in this case, XHTML containing embedded MathML (math markup language for the web). If the resulting .xml file is opened in Internet Explorer with a screen reader, sophisticated mathematical statements will be intelligible when the free MathPlayer add-in has been installed from \ <http://www.dessci.com/en/products/mathplayer/download.htm>



# Programming

Press Tab to indent the current line of text, or Shift+Tab to outdent it. If multiple lines of text are selected, these commands are applied to all of them. The Trim Blanks command, Control+Shift+Enter, removes all indentation and trailing spaces at once, as well as removing more than two consecutive blank lines (when multiple lines are selected).

Press Alt+I to hear the number of indentation levels of the current line. Alt+Shift+I toggles a mode in which you are alerted to changes in indentation level, such as when using the up and down arrow keys. EdSharp will say how many levels in or out the indentation has changed. This mode also reverses the roles of the Enter and Shift+Enter keys.

When Indent Mode is off, you can start a new line of text with the same indentation as the current one by pressing Shift+Enter. By default, an indentation unit is one tab character. This may be changed with the Configuration Options command, Alt+Shift+C.

To go to the first character of the current line after any indentation, press Alt+Home. To go to the last non-white space character, press Alt+End.

Use the Infer Indent command, Alt+RightBracket, to hear what indent unit the current document seems to be using. EdSharp looks at the first line that starts with a space or tab character. If this key is pressed again without moving the cursor, that sequence of space or tab characters is configured as EdSharp's IndentUnit setting. This makes it easy to use the same indentation style as a file you have opened.

Press Control+B to go to the next code block, or Control+Shift+B to go to the previous one. A block is defined as lines with the same or greater indentation/nesting. Control+I and Control+Shift+I have a similar purpose, but they move to the next or previous change in indentation. EdSharp skips blank or commented lines with these commands.

Control+I will stop at a nested block, whereas Control+B will not, since it

continues past lines with greater indentation. For example, if the cursor is inside a loop block, then Control+I will go to the line at the closing of the loop where a lower level of indentation resumes. In Ruby, this would be the line with the word "end". In Python, it would be the first line of code following the loop, since the change in indentation, itself, indicates the end of the loop.

The related query commands, Alt+B and Alt+I, help you understand code groupings without moving the cursor. They provide additional information when pressed a second time in a row. Alt+B says the rest of the current block, beginning at the current line. When toggled with a second press, it says the whole block, including lines prior to the current one, if any. Alt+I says the indentation level of the current line. When toggled, it reads the text of the preceding line with less indentation, which is typically the statement that introduced the current block, e.g., an if, for, or while statement. These commands are best learned by experimenting with familiar code.

The Quote and Unquote commands, Control+Q and Control+Shift+Q, may be used to add or remove comment symbols at the start of lines. The default quote prefix may be changed from > to a comment sequence appropriate for the language in use, e.g., ' for Visual Basic, \* for Xbase, ; for AutoIt, or # for Ruby.

Curly brace characters delimit code structures in a number of languages. Press Control+Shift+RightBracket to find the matching right brace (}) character from the current location. Press Control+Shift+LeftBracket for the matching left brace ({) instead. Press Alt+Shift+RightBracket to hear the number of unmatched left braces before the cursor and right braces after. Different brace characters may be configured, e.g., angle brackets (<>) for editing HTML or XML. If the cursor is on a brace-type character when issuing one of these commands, i.e., one of {}<>, then EdSharp uses that character and its opposite when searching, regardless of the current setting. In addition, Control+Shift+ (think of the > symbol) goes to the matching end tag of an HTML element, and Control+Shift+, goes to the start tag.

Two commands specifically aid programming in the Python language with speech. This language requires indentation for subordinate code blocks, and the indentation, itself, rather than words or punctuation, is how such structure is specified. The indentation is helpful to sighted programmers, and often to users of large print or braille, but generally inefficient for speech users, since

such access tends to be serial rather than two dimensional in nature. The PyBrace command (Alt+Shift+LeftBracket) converts indented structure to a form explicitly indicated by opening and closing braces, similar to C-like languages. The PyDent command (Alt+LeftBracket) does the reverse, converting from PyBrace to indented format that is understood by the Python interpreter (using the current IndentUnit setting).

The PyBrace and PyDent formats include comments that indicate when a block has closed , e.g., "# end for" on the line after a "for" block. If text is selected, these commands replace it with the alternate format; otherwise, they create a new editing window so the original file is still available.

A scripting language allows a program to be run as a text file associated by extension with its interpreter, e.g., .py for Python, .au3 for AutoIt, or .rb for Ruby. Press F5 to run the current file with its associated interpreter. If the current file name has a complete path, EdSharp saves to disk before running the file to ensure the latest version is being used. Otherwise, EdSharp saves to a file in a temporary folder and runs that file.

The Alt+F5 command prompts for a command to run and speaks its standard output. The path of the current file may be passed via the syntax described for EdSharp's Import and Export capability. The command remembers its previous value, and may be adjusted each time it is run. Use the Review Output command, Alt+Shift+F5, to open a new editing window containing the output produced by the last command.

Use the Compile command, Control+F5, for a programming language that involves compiling source code to binary form. For example, a C# program in a .cs file may be compiled to a .exe file. This command may also be used for interpreters that report syntax errors via the standard output or standard error streams.

These tool commands typically begin with the file name of the compiler or interpreter. Any parameters may be specified thereafter. If the token %SourceDir% is included, EdSharp temporarily changes to the directory containing the source file before running the tool.

The first line and column position mentioned in the output, if any, is assumed to

be the position of a compilation error in the source code. EdSharp uses the `JumpPosition` setting to find the position in the output based on a regular expression. The regular expression should be defined so that the first number of a matching string is the line number and the second number, if any, is the column number. EdSharp automatically jumps to that position. It is also saved so that the `Jump Again` command, `Alt+J`, returns there.

Another regular expression may be configured for navigating among routines in source code. The `NavigatePart` setting is used by `Alt+PageDown` and `Alt+PageUp` to go to the next or previous function, method, or class definition. The `Go to Part` command, `Alt+Shift+G`, lets you pick one of these locations from a list.

Thus, the `Compile` command, `Control+F5`, combines debugging steps efficiently by compiling, saying output without a modal message box, and automatically jumping to the first error position, if found in the output. The output spoken may be abbreviated by means of a regular expression setting that specifies the pattern of text to remove. The `Pick Compiler` command, `Control+Shift+F5`, lets you conveniently configure the `CompileCommand`, `AbbreviateOutput`, `JumpPosition`, `NavigatePart`, and `QuotePrefix` settings for a particular compiler or interpreter. EdSharp offers settings for the following languages: Boo, C#, HTML, Java, JAWS Script, JScript .NET, LaTeX, Perl, PHP, PowerBASIC, PowerShell, Python, Ruby, and Visual Basic .NET.

The name of a tool to be run should either include its directory location or be available on the Windows search path. This may be adjusted by editing the `Path` environment variable in the `Advanced` tab page of the `System` applet in `Control Panel`. If the tool is a long file name enclosed in quotes then either prefix the command line with the `@` symbol or enclose the whole thing in quotes. This is necessary to prevent `.ini` file manipulation functions of Windows from losing the opening quote before the tool.

For HTML, the HTML Tidy utility is configured by default and distributed with EdSharp. After eliminating coding errors found with `Control+F5`, use `Alt+Shift+E` to export to a target file containing clean HTML. More information is available at <http://tidy.sourceforge.net>

For PowerBASIC, a batch file is needed (in the EdSharp program folder),

which refers to the default location of PowerBASIC for Windows version 10.0. The path to the JAWS script compiler is also hard coded for the latest version. JAWS scripting is additionally supported by EdSharp's own scripts: Control+I is a hot key for inserting the path to the user script folder, and Control+Shift+I is for the All Users script folder, when focus is in the Open or Save Dialog of EdSharp.

Compiler settings are stored in the [Compilers] section of the EdSharp.ini file. Only current compiler settings appear in the configuration options dialog, Alt+Shift+C. Other settings may be edited, however, using the Manual Options command, Alt+Shift+M. You can adjust command line parameters of configured compilers, or add others. Installing a new version of EdSharp does not change existing compiler settings.

## Scripting Add-Ins

Almost the complete object model of the EdSharp application has been exposed to add-in code in the JScript .NET language, explained at [http://msdn2.microsoft.com/en-us/library/t895bwkh\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/t895bwkh(VS.80).aspx)

JScript.NET is a version of JavaScript with access to the huge library of the .NET Framework. In EdSharp, JScript code may be used in the Evaluate Expression command (Control+Equals), Replace Tokens command (Control+Shift+Equals), and Paste Snippet command (Alt+V). Stand-alone JScript executables may also be created with the JScript .NET compiler (jsc.exe), which is included with free .NET developer tools.

Using the Compile command (Control+F5) is the best way to debug JScript code even if you want to use it as an add-in rather than stand-alone executable. This is because the JScript compiler provides error information that is not available when add-in code fails to execute due to syntax errors. When writing code in this way, you will probably want to import one or more namespaces to abbreviate .NET references, e.g., by copying statements at the top of the Eval.js file in the EdSharp program folder. Comment out such import statements in the debugged, add-in version of the code, since EdSharp already calls them internally.

The EdSharp object model includes a hierarchy of classes corresponding to the overall application, multiple document interface (MDI) frame, MDI child windows, and RichTextBox (RTB) within each window. Typically, a script will manipulate text in the current RTB control. The Frame property of the App class refers to the single MDI frame. The Child property of that frame object refers to the active MDI child. The RTB property of that child object refers to the current editing control.

Thus, a JScript routine might start by creating one or more object variables as follows:

```
var frame = App.Frame
var child = frame.Child
var rtb = child.RTB
```

By convention, .NET properties are initially capitalized, whereas field and local variables are not. Methods of the frame object can invoke menu items, e.g., a new editing window could be created with the following statement:  
`frame.menuFileNew.PerformClick()` Methods and properties of an RTB (RichTextBox) object are explained at [http://msdn2.microsoft.com/en-us/library/system.windows.forms.richtextbox\\_members\(vs.80\).aspx](http://msdn2.microsoft.com/en-us/library/system.windows.forms.richtextbox_members(vs.80).aspx)

EdSharp also adds some methods and properties in its inherited version of the RichTextBox class, e.g., the ReplaceRange method for replacing text between two points in the current document. Other EdSharp classes provide convenient scripting methods, e.g., Dialog.Pick gets a user choice from a listbox and Util.String2File saves a string of text to a file on disk.

These classes will be further documented based on questions received. At present, the best way to learn them is to examine code in sample .js snippet files and the main EdSharp.cs program file, which implement behavior you experience when running the application. Although the .cs code is in the C# language, its syntax is similar to JScript, and the names of classes, methods, and properties are the same.

## Miscellaneous

Extra speech messages may be toggled off -- or reactivated -- with Control+Shift+X. When off, such messages are redirected to a text file in the EdSharp data directory called Speech.log, which may be examined in an editing window with Alt+Shift+X. This file is initialized when EdSharp starts, and the Extra Speech setting is remembered from the previous session.

With the optional JAWS scripts, you can toggle a speech setting of reading all or no punctuation using JAWSKey plus the grave accent at the top left of the main keypad (U.S. keyboard). All punctuation is useful when reading carefully for details whereas no punctuation is useful when reading quickly for concepts.

Press Control+W to turn word wrap on, or Control+Shift+W to turn it off. Use the Guard Document command, Control+F7, to make the document read-only, preventing accidental modifications. Control+Shift+F7 drops this protection. Wrap and guard settings are restored the next time a file is opened.

EdSharp checks whether the file in the current editing window has been modified by another program since being loaded from disk. If so, you are prompted whether to open it again (like what Alt+O does manually). If you answer No, version checking on the current file stops until you save or reload it.

Use the Repeat Line command, Control+Y, to make a copy of the current line directly below it. The cursor is placed at the start of the new line. This is useful for creating a new line of text by editing a previous line that is similar.

Press Control+Equals to evaluate either the current line or selected text as an expression in the JScript.NET language. This command is useful for mathematical calculations. For example, the following algebra calculates the cumulative total of an initial 100 deposit compounded for 10 years at an annual rate of 5% interest:\

```
var interest = 1.15 var deposit = 100 var years = 10
Math.Pow(interest, years) * deposit 162.889462677744
```



The result, about \$163, was placed on the line below the previously selected text, and the cursor was placed at the start of that line.

The Transform Files command, Alt+Equals, applies a saved set of search and replace tasks to one or more files -- typically to massage data or formatting in predictable ways. EdSharp prompts for the job file containing the regular expressions to apply. Each task is defined by three lines: (1) a comment explaining the operation, (2) the search expression, and (3) the replacement expression. A blank line separates each task.

Before invoking this command, the current editing window should contain the list of files to process, one per line. Such a list could be typed manually or generated via the Path List command (Control+Shift+P). If a file does not include a leading path, the prior one is assumed.

An intervening dialog lets you test what changes would occur without actually performing them. In either case, you can subsequently use the Review Output command, Alt+Shift+F5, to examine the change log.

Here is the content of a sample transform job that defines two tasks:\

```
``` [Begin Content of TrimLine.job] Remove leading space or tab characters
from each line (\A\n)( |t)+ $1
```

```
Remove trailing space or tab characters from each line ( |t)+(\r|Z) $2 [End
Content of TrimLine.job] ```
```

Press Alt+Shift+Semicolon to insert the current time and date at the cursor position. The configurable format defaults to text like the following:\ 5:43 AM
Sunday, May 13, 2007

Configuration settings let you adjust the date and time formats according to templates explained at [http://msdn.microsoft.com/en-us/library/97x6twysz\(VS.80\).aspx](http://msdn.microsoft.com/en-us/library/97x6twysz(VS.80).aspx) Specify a setting of 0 for the date or time component to be excluded.

Press Control+Shift+Semicolon to calculate and insert a date at the cursor position. EdSharp prompts for the year, month, week, and day. Specify the year

and week as numbers, the latter being optional. The month and day may be either numbers, words, or abbreviations. Here are two examples.

```
Task: Calculate the date of Thanksgiving -- the 4th Thursday  
of November Year: 2007 Month: November Week: 4 Day: Thursday  
Result: Thursday, November 22, 2007
```

```
Task: Calculate which day is Christmas Year: 2007 Month: Dec  
Week: Day: 25 Result: Tuesday, December 25, 2007
```

Use the HTML Format command, Control+H, to convert the current document to HTML in a new window. Structured text is converted to equivalent HTML with a table of contents and section headings. By examining the files EdSharp.txt and EdSharp.htm in the EdSharp program folder, you can compare the text before and HTML after such a conversion.

In a structured text file, the first line is assumed to be the title of the resulting web page. Lines beginning with a web or email address are converted to links. Typically, you would use this command after creating sections with Control+Enter and generating a table of contents with Alt+Shift+T.

Use the Text Convert command, Control+T, to convert a list of files in the current editing window to text files with the same names except for .txt extensions. Press Control+Shift+T to convert and append source files to text in a new document instead.

The Web Download command, Alt+Shift+W, lets you pick one or more files to download from a page whose address you specify. If Internet Explorer is open, EdSharp uses the value in its address bar as the default. Each item of the resulting checked listbox shows both the clickable text of the url and its target file name. Press Spacebar to toggle the checked state of an item. After picking files, you are prompted for the target folder on disk. If the URL of a link does not end in a valid file name, EdSharp creates a file name for the target on disk based on other characters in the URL. If a file with the same name already exists, a unique name is created by adding a numeric suffix, e.g., page_001.htm, page_002.htm, etc.

To make navigation more flexible and efficient in a listbox with many items, EdSharp adds the following features to its list-based dialogs.

Control+J prompts for text within an item, going to the first match if a new search, or the next match if the previous value is accepted. Alt+J goes to the next match without prompting for a value. The item with focus when the dialog is closed -- but not canceled -- becomes the current item the next time that the same list dialog is invoked (you are notified when it is not the first item). The Jump value of that dialog is also remembered.

Control+F sets a filter to restrict what items are shown via wildcards (*to match any sequence of characters or ? to match a single one*). For example, you could browse replace-related commands in the Alternate Menu, Alt+F10, by pressing Control+F after invoking that list and then entering replace* as the filter expression. Control+Shift+F clears the filter so all items are shown again. The order of items may also be changed: Alt+A for alpha order, Alt+Shift+A for reverse alpha order, Alt+D for default order, or Alt+Shift+D for reverse default order.

Multiple commands support flexible checking or unchecking in a checked listbox such as the Web Download dialog. Press Space to toggle the checked state of the current item, Control+A to check all items, or Control+Shift+A to uncheck all. Press Shift+DownArrow for check and Next, or Shift+UpArrow for check and Previous. Press Shift+End for check to Bottom, or Shift+Home for check to Top. Shift+NumPad5 checks the current item. F8 marks the start of a checking operation, completed with Shift+F8.

Adding the Alt modifier key performs the same action except for unchecking rather than checking. Thus, Alt+Shift+NumPad5 unchecks the current item, Alt+Shift+Home unchecks to the top of the list, Alt+Shift+End unchecks to the bottom, Alt+Shift+DownArrow unchecks en route to the next item, and Alt+Shift+UpArrow unchecks en route to the previous. F8 then Alt+Shift+F8 unchecks items in that range.

Other arrow keypad actions navigate among checked items. Control+Home goes to the top checked item, and Control+End goes to the bottom one. Control+DownArrow goes to the Next , and Control+UpArrow goes to the previous.

Shift+Space tells you what items are currently checked. Alt+A says the address of the current item in the list, e.g., 11 of 42.

Press Alt+Shift+C to adjust configuration options of EdSharp through a dialog. Each option has a unique access key in its label, so you can jump directly to it with an Alt plus letter combination.

You can configure whether EdSharp's application window is maximized at startup, and whether an editing window is word wrapped when created -- the default is Yes for these options. When a file is saved without giving it an extension, .rtf is added as a configurable default. If a file would be overwritten, the original may be optionally saved with .bak added (default is No). The OpenPrevious option determines whether files open at the end of the previous session are automatically opened at the start of the next one (default is No). Another option limits the number of files shown with the Recent Files command, Alt+R (default is 100).

The HardPageAddress option determines whether the Address command, Alt+A, gives a page number instead of document percentage (default is No). A form feed character specifies a hard page break. It is part of the sequence inserted by the Section Break command, Control+Enter, which is configurable via the SectionBreak option of the configuration dialog. Use the Control+PageDown and Control+PageUp commands to navigate by page. Pressing Alt+A a second time in a row gives the alternate type of address information, so you can still get a page number without changing the HardPageAddress setting.

The ViewLevels option is a list of file extensions and associated numbers that specify how EdSharp should handle conversions when files are opened through Windows Explorer or by the Recent Files command. For example, this setting includes "md:0 pdf:1" by default. If you associate the .pdf extension with EdSharp (e.g., via the shortcut in the EdSharp program group of the Start Menu), then EdSharp will automatically convert a PDF to text when opened. On the other hand, the 0 value associated with the .md extension means that no conversion will automatically be attempted on such a file type (Markdown format).

The UseIndentModeDefault setting determines the state of indent mode when a new document window is opened. This mode may be toggled on a per-window basis with Alt+Shift+I. The configuration setting determines whether it is initially on or off when a file is opened (default is No).

The YieldEncoding setting determines the character encoding EdSharp will use when opening a file from disk. EdSharp ignores this setting if the file has a .rtf extension indicating rich text format, or has an initial byte order mark (BOM) indicating Unicode format (e.g., UTF-8 or UTF-16). An encoding may be indicated by either its name or number. A list of those available may be found by choosing the Other option in the Yield Encoding command, Alt+Shift+Y, or Export Format command, Alt+Shift+E. If no YieldEncoding setting is configured, EdSharp uses the default encoding configured in the regional settings applet of Windows Control Panel. Typically, the setting is Western European. Use the Status command, Alt+Z, pressed twice in order to check the encoding that EdSharp used to open the current document. EdSharp will save the document with the same encoding.

A separate dialog invoked with Alt+Shift+Equals is available for setting the font and color of text in an editing window. The default font was chosen for friendliness to low vision users, and you can adjust this subjective choice. Use the Manual Options command, Alt+Shift+M, to directly edit the main configuration file, EdSharp.ini, located in the EdSharp data folder. This is a subfolder of the Windows path to Application Data, typically named something like \C:\users\UserName\appdata\roaming\EdSharp\

To change key assignments, edit the Keys section of the configuration file, e.g., the line \Replace=Control+R could be changed to \Replace=Control+H. Since Control+H is assigned to the HTML Format command by default, however, EdSharp will alert you to a conflict when loading unless that command is also reassigned or detached from a key as follows: \HTML
Format=None

A command without a hot key may still be invoked via the regular menu system or Alternate Menu (Alt+F10). The terms used to identify available keys are listed on the following web page: [http://msdn2.microsoft.com/en-us/library/system.windows.forms.keys\(vs.80\).aspx](http://msdn2.microsoft.com/en-us/library/system.windows.forms.keys(vs.80).aspx)

Certain configuration options are associated with the current compiler rather than being global. Specifically, favorites, bookmarks, and user-defined tokens apply to the current compiler (picked via Control+Shift+F5), so you can work with items more relevant to each coding project. The Reset Configuration command, Alt+Shift+0, lets you easily remove custom settings and restore

defaults of EdSharp. This command lets you choose whether to reset the main configuration, current compiler configuration, or create a new compiler configuration. The New choice prompts for the compiler name, command line, AbbreviateOutput, NavigatePart, QuotePrefix, and ExtensionDefault settings.

A compiler configuration file is stored in the EdSharp data folder in a file having the compiler name and a .ini extension. For example, if you created settings for the "Delphi" compiler, EdSharp would create a Delphi entry in the Compilers section of EdSharp.ini, and then store related favorites, bookmarks, and user-defined tokens in Delphi.ini. Press Alt+0 to query the current compiler and directory.

Press F4 to activate an editing window from a list of those currently open. Press Control+F4 to close the current window, or Control+Shift+F4 to close all windows except the current one. EdSharp windows may be visually organized according to common MDI (multiple document interface) patterns. The Window menu includes the following commands: Arrange Icons, Alt+F11; Cascade, Control+F11; Tile Horizontal, Alt+Shift+F11; and Tile Vertical, Control+Shift+F11.

Use the Alternate Menu command, Alt+F10, to execute a command from a single, alphabetized list.

The Context Menu command, Shift+F10, lets you choose an action to perform on the current file based on those available for its type/extension (in the Windows registry). Also included is the OpenWith action, by which a default program may be associated with files of this type. The Send To Menu, Control+F10, lets you choose among SendTo shortcuts (installed by various applications) to perform on the current file.

Online Help

Press Alt+Shift+H for a summary of EdSharp hot keys. Press F1 to load this documentation, or Alt+F1 to simply confirm the version. Control+F1 toggles a Key Describer mode in which pressing a key describes its action. If you switch to another application window, the mode is automatically turned off.

Use the Environment Variables command, Control+E, to review or change such

settings of Windows. Choose those of the current process, user, or system as a whole. Jump quickly to a particular variable based on its initial letter, e.g., Alt+P for the PATH setting that determines where Windows searches for an executable file that is not found in the current directory. Changes to process settings affect the current session of EdSharp, but not the next time it is run. User settings take effect when you log in again. System settings take effect when you restart the computer.

The Burn to CD command, Alt+Shift+B, operates on a path list in the current document. Each file or folder path should be placed on a separate line. If a parent folder is not specified, the last one in the list is used. The default parent folder is the current directory. For example, the following list specifies the EdSharp program folder and two test files from the temp folder:\

```
C:\Program Files\EdSharp C:\temp\test.txt test.doc
```

You can create a path list manually, or use the Path List command, Control+Shift+P, to generate one for you. The paths found are burned to the default CD drive. The CD may be blank or include content already. The new content is added to it. The CD must be writable. At this time, DVDs do not work, only CDs.

Use the Elevate Version command, F11, to update EdSharp to the current version available at http://EmpowermentZone.com/EdSharp_setup.exe ###
Hotkey Summary The following are EdSharp commands listed in related groups.

Launch EdSharp=Alt+Control+E, Launch or activate the EdSharp application via a Windows desktop shortcut `` Documentation=F1, Open Documentation in web browser About=Alt+F1, Display version and release date History of Changes=Shift+F1, Display list of fixes and improvements Key Describer=Control+F1, Toggle a mode in which pressing a key describes its action Alternate Menu=Alt+F10, Present all commands in a single, alphabetized list Context Menu=Shift+F10, Pick a command from those available to Windows Explorer for the current file extension SendTo Menu=Control+F10, Pick a command from those available as Windows "Send To" options

Select All=Control+A, Select all text Unselect All=Control+Shift+A, Clear text selection

Select Chunk=Control+Space, Select contiguous sequence of non-blank characters at cursor, or select the next chunk if a selection already exists Say Selected=Shift+Space or JAWSKey+Shift+DownArrow, Say selected text, or spell if repeated Say Chunk=Shift+BackSpace, Say chunk at cursor

Start Selection=F8, Mark starting point of text to be selected Complete Selection=Shift+F8, Select text from starting point to cursor
Reselect=Control+Shift+F8, Reselect between previous start and end positions
Go to Start of Selection=Alt+Shift+F8, Return to start position of selection
Copy All=Control+F8, Copy all text to clipboard Read All=Alt+F8, Say all text (without moving cursor)

Say Address=Alt+A, Say line, column, and percent position of cursor Say Block=Alt+B, Say the rest of the current code block, or the whole block if repeated Say Indent=Alt+I, Say the indentation level of the current line, or the preceding line with less indentation if repeated Say Yield=Alt+Y, Say number of characters, words, and lines in all or selected text Say Status=Alt+Z, Say whether current file has been modified since last save to disk, or say its character encoding if repeated Say Clipboard=Alt+Apostrophe, Say clipboard text, or spell if repeated Say Time=Alt+Semi-colon, Say current time and date Insert Time=Alt+Shift+Semi-colon, Insert current time and date Calculate Date=Control+Shift+Semi-colon, Calculate and insert date

Configuration Options=Alt+Shift+C, Adjust configuration options through a dialog Set Default Font and Color=Alt+Shift+Equals, Set default font and color for editing window Manual Options=Alt+Shift+M, Adjust options by directly editing the main configuration file Reset Configuration=Alt+Shift+0, Revert to default options, or define a new compiler configuration

Copy=Control+C, Copy selected text to clipboard, or copy current line if no selection Copy Append=Alt+C, Append selected text to clipboard, or append current line if no selection Copy Rich Text=Control+Shift+C, Copy selected text with formatting to clipboard Cut=Control+X, Cut selected text to clipboard, or cut current line if no selection Cut Append=Alt+X, Cut and append selected text to clipboard, or cut and append current line if no selection

Paste=Control+V, Paste text from clipboard Paste File=Control+Shift+V, Insert another file at cursor position Append from Clipboard=Alt+7, Toggle a mode in which text copied to the clipboard is also saved to a file Undo=Control+Z, Undo the last editing action Redo=Control+Shift+Z, Redo the last action that was undone

Save Snippet=Alt+S, Save all or selected text to a snippet file Invoke Snippet=Alt+V, Pick snippet file to paste or execute View Snippet=Alt+Shift+V, Pick snippet file to view or edit

Yield with Regular Expression=Control+Shift+Y, Count parts of text matching a regular expression Extract with Regular Expression=Control+Shift+E, Extract text matching a regular expression, putting matches in a new window Replace with Regular Expression=Control+Shift+R, Search and replace regular expression in all or selected text Replace=Control+R, Search and replace string in all or selected text

File Find=Alt+Shift+F, Open file from list of files containing a search string Forward Find=Control+F, Search forward for string in all or selected text Reverse Find=Control+Shift+F, Search backward for string Forward Find with Regular Expression=Control+F3, Search forward for regular expression in all or selected text Reverse Find with Regular Expression=Control+Shift+F3, Search forward for regular expression in all or selected text Forward Find at Cursor=Alt+F3, Search forward for chunk or selected text Reverse Find at Cursor=Alt+Shift+F3, Search backward for chunk or selected text Forward Find Again=F3, Search forward for next match Reverse Find Again=Shift+F3, Search backward for previous match

Word Wrap=Control+W, Word wrap lines Unwrap=Control+Shift+W, Unwrap lines Guard Document=Control+F7, Make document read-only No Guard=Control+Shift+F7, Clear read-only status

Extra Speech Toggle=Control+Shift+X, Toggle extra speech messages on or off, redirecting to Speech.log file Extra Speech Log=Alt+Shift+X Open speech.log file in a new window

Go to Percent=Control+G, Go to percentage point in document Go to Percent Again=Alt+G, Repeat Go command Jump to Line=Control+J, Jump to line

number or to line, column position Jump to Line Again=Alt+J, Repeat Jump command

Set Bookmark=Control+K, Set bookmark at cursor position Clear
Bookmark=Control+Shift+K, Clear bookmark at cursor position Go to
Bookmark=Alt+K, Go to bookmark in current file

Set Favorite=Control+L, Add current file to the list of favorites Clear
Favorite=Control+Shift+L, Clear current file from the list of favorites List
Favorites=Alt+L, Open a file from the list of favorites Recent Files=Alt+R,
Open a file from the list of those recently used

New=Control+N, Open a new editing window New from
Clipboard=Control+Shift+N, Open a new editing window containing
clipboard text

Open=Control+O, Open file Open Other Format=Control+Shift+O, Open file
in another format and convert it to text Open Again=Alt+O, Reload the current
file from disk

Properties=Alt+Enter, display Windows properties dialog for current file
Save=Control+S, Save Save As=Control+Shift+S, Save As Save
Copy=Alt+Shift+S, Save copy of document using a different name Export
Format=Alt+Shift+E, Export document to another format

Print=Control+P, Print current file Mail Body=Control+M, Mail current file as
body of an email message Mail Attachment=Control+Shift+M, Mail current
file as an email attachment Burn to CD=Alt+Shift+B, Burn list of files or
folders to a CD Web Download=Alt+Shift+W, Pick files to download from a
web page or the current document

Run=F5, Execute current file, based on its extension Run at Cursor=Shift+F5,
Execute a web URL or email address at cursor position or in selected text
Prompt Command=Alt+F5, Prompt for a command line to execute and say its
standard output Review Output=Alt+Shift+F5, Open standard output of last
prompt or compile command in a new editing window Compile=Control+F5,
Compile source code, say output, and jump to error position Pick
Compiler=Control+Shift+F5, Pick a compiler or interpreter from the list of

those configured Say Compiler=Alt+0, Say current compiler and folder Go to Folder=Control+0, Go to folder containing recent or favorite files Go to Special Folder=Control+Shift+0, Go to special folder of Windows Go to Environment=Control+Shift+G, Go to interactive environment of current compiler

Spell Check=F7, Spell check all or selected text Thesaurus=Shift+F7, Look up synonyms for word at cursor

Say Path=Alt+P, Say full path of current file Path to Clipboard=Alt+Shift+P, Copy full path of current file to clipboard Path List=Control+Shift+P, Generate a list of files in a new editing window

Special Character=F2, Insert character indirectly by specifying its Unicode value Quote=Control+Q, Add prefix sequence to current or selected lines Unquote=Control+Shift+Q, Remove prefix sequence from current or selected lines Join Lines=Control+Shift+J, Word wrap lines in all or selected paragraphs Hard Line Break=Control+Shift+H, Set the maximum width of lines in all or selected text

Upper Case=Control+U, Convert current or selected characters to upper case Lower Case=Control+Shift+U, Convert current or selected characters to lower case Proper Case=Alt+U, Convert current or selected characters to proper case Swap Case=Alt+Shift+U, Convert lower case characters to upper case, and vice versa Yield Encoding=Alt+Shift+Y, Render all or selected text based on a character encoding

Format Code =Control+4, Arrange indentation and other stylistic conventions in a C-like language Repeat Line=Control+Y, Copy current line below it Evaluate Expression=Control+Equals, Evaluate current line or selected text as a JScript.NET expression and copy the result below Replace Tokens=Control+Shift+Equals, Swap user-defined tokens with their computed results in all or selected text Transform Files=Alt+Equals, Apply a set of search and replace tasks to a list of files in the current window Trim Blanks=Control+Shift+Enter, Trim leading and trailing blanks from the current or selected lines, and remove more than two consecutive blank lines

End Character=Alt+End, Go to last non-blank character of line and read it

Home Character=Alt+Home, Go to first non-blank character of line and read it
Next Word=Control+RightArrow, Go to next word and read it Prior
Word=Control+LeftArrow, Go to previous word and read it Next
Chunk=Alt+RightArrow, Go to next chunk and read it Prior
Chunk=Alt+LeftArrow, Go to previous chunk and read it Next
Sentence=Alt+DownArrow, Go to next sentence and read it Prior
Sentence=Alt+UpArrow, Go to previous sentence and read it Next
Paragraph=Control+DownArrow, Go to next paragraph and read it Prior
Paragraph=Control+UpArrow, Go to previous paragraph and read it

Delete Right=Control+Shift+Delete, Delete from cursor to end of line Delete
Left=Control+Shift+Backspace, Delete from cursor to start of line Delete
Down=Alt+Shift+Delete, Delete from cursor to bottom of file Delete
Up=Alt+Shift+Backspace, Delete from cursor to top of file Delete
Line=Alt+Backspace, Delete current line Delete Hard Line=Control+D, Delete
line ending in hard line break Delete Paragraph=Control+Shift+D, Delete past
one or more blank lines Delete File=Alt+Shift+D Delete current file on disk
Rename=Alt+Shift+R Rename current file on disk

Next Section=Control+PageDown, Go to next section Prior
Section=Control+PageUp, Go to Prior Section Go to Section=F6, Go to
section in body from topic in table of contents Go to Contents=Shift+F6, Go to
topic in table of contents from section in body Search for Topic=Control+F6,
Search for a topic based on text in its heading Search for Topic Again=Alt+F6,
Search again for the next matching topic

Topic=Alt+T, Say topic of current section Text Contents=Alt+Shift+T,
Generate and prepend a table of contents to the current document Section
Break=Control+Enter, Insert a section break at the cursor position

HTML Format=Control+H, Convert current document to HTML in a new
window Text Convert=Control+T, Convert other formats to text files with the
same name except for a .txt extension Text Combine=Control+Shift+T, Convert
other formats to text and combine them in a new editing window

Justify=Alt+Shift+J, Set justification of cursor or selected text
Style=Alt+Shift+Slash, Set style of cursor or selected text
Baseline=Alt+Shift+6, Set vertical alignment of cursor or selected text Set

Selection Font=Alt+Shift+Dash, Set font of cursor or selected text
Next Alignment=Control+RightBracket, Go to next change in justification
Prior Alignment=Control+LeftBracket, Go to previous change in justification
Next Style=Control+Slash, Go to next change in style
Prior Style=Control+Shift+Slash, Go to previous change in style
Next Baseline=Control+6, Go to next change in baseline
Prior Baseline=Control+Shift+6, Go to previous change in baseline
Next Font=Control+Dash, Go to next change in font
Prior Font=Control+Shift+Dash, Go to previous change in font
Say Font=Alt+Dash, Say current font and color
Say Styles=Alt+Slash, Say current justification and styles

Infer Indent=Alt+RightBracket, Infer the indent unit of the current document, or configure EdSharp accordingly if repeated
Toggle Indentation=Windows+Grave, Toggle announcement of indentation by JAWS
Indent Mode=Alt+Shift+I, Toggle auto indent with Enter, and announcement of indentation changes
Enter New Line=Enter, Start new line at left margin
Indent New Line=Shift+Enter, Start new line with same indentation as current one
Indent New Line Prior=Alt+Shift+Enter, insert prior line with same indentation as current one
Indent=Tab, Indent current line or selected text by one unit
Outdent=Shift+Tab, Reduce indentation of current or selected lines by one unit
Align=Alt+Shift+A, Adjust indentation of current or selected lines according to prior line
Next Block=Control+B, Go to the next block of code, having the same or less indentation
Prior Block=Control+Shift+B, Go to the previous block of code, having the same or less indentation
Next Indent=Control+I, Go to the next change in indentation
Prior Indent=Control+Shift+I, Go to the previous change in indentation
Right Brace=Control+Shift+RightBracket, Search forward for matching right brace character
Left Brace=Control+Shift+LeftBracket, Search backward for matching left brace character
End Tag=Control+Shift+Period, go to closing tag of HTML element
Start Tag=Control+Shift+Comma, Go to opening tag of HTML element
Next Part=Alt+PageDown, Go to next match of NavigatePart setting
Prior Part=Alt+PageUp, Go to previous match of NavigatePart setting
Go to Part=Alt+Shift+G, Pick a part to go to

Order Items=Alt+Shift+O, Sort items alphabetically in all or selected text
Reverse Items=Alt+Shift+Z, Reverse order of all or selected items of text
Keep Unique Items=Alt+Shift+K, Discard repetitive items in all or selected

text Number Items=Alt+Shift+N, Insert numbers at the start of items in all or selected text List Different Items=Alt+Shift+L, Compare two lists and put non-overlapping items in a new window Query Common Items=Alt+Shift+Q, Compare two lists and put overlapping items in a new window

PyDent=Alt+LeftBracket, Convert from PyBrace format, or reformat typical Python code, using the IndentUnit setting and adding comments at ends of blocks PyBrace=Alt+Shift+LeftBracket, Convert from PyDent format, or reformat typical Python code, using braces instead of indentation and adding comments at ends of blocks

Explorer Folder=Alt+Backslash, Open Windows Explorer in the EdSharp program folder, data folder, or current folder Command Prompt=Control+Backslash, Open a command prompt in the EdSharp program folder, data folder, or current folder Environment Variables=Control+E, Change Windows environment variables for the current process, user, or system

Next Window=Control+Tab, Cycle to next editing window Prior Window=Control+Shift+Tab, Cycle to previous editing window Windows Open=Shift+F4, Say titles of current editing windows Current Windows=F4, Activate an editing window from a list of those currently open Close Window=Control+F4, Close current editing window Close All but Current Window=Control+Shift+F4, Close all editing windows except the current one Exit EdSharp=Alt+F4, Exit the EdSharp application

Arrange Icons=Alt+F11, Arrange open windows Cascade=Control+F11, Cascade open windows Tile Horizontal=Alt+Shift+F11, Tile open windows horizontally Tile Vertical=Control+Shift+F11, Tile open windows vertically

Elevate Version=F11, Download latest EdSharp version and run installer (after confirming)

With JAWS scripts: Toggle Punctuation=JAWSKey+Grave Accent, Toggle JAWS voice between all and no punctuation Voice Louder=Alt+Grave, Increase JAWS voice volume by 5% Voice Softer=Alt+Shift+Grave, Decrease JAWS voice volume by 5% Voice Faster=Control+Grave, Increase JAWS voice rate by 5% Voice Slower=Control+Shift+Grave, Decrease JAWS voice

rate by 5% Insert Script Path=Control+I, Insert JAWS script path in Open or Save Dialog Insert All Users Path=Control+Shift+I, Insert JAWS All Users path in Open or Save Dialog

Development Notes

For the technically curious, I developed EdSharp with the C# programming language from\ <http://msdn.microsoft.com/en-us/vcsharp/default.aspx>

The executable, EdSharp.exe, may be recompiled with the batch file, compileEdSharp.bat, located in the EdSharp program folder. It assumes that the C# 4.0 command-line compiler, csc.exe, is in the default location.

To replicate the folder structure I use during development, copy the EdSharp program folder and all its subfolders to\ C:\EdSharp

Contributors

Thanks go to Jim Homme for contributing an improved set of HTML snippets. He also contributed the Sounds4Stuff scheme for JAWS, which may be installed via Settings Packager.

I thank Jaffer for contributing C++ and PHP snippets.

Third Party Utilities

PDF conversions use the open source software available at\ <http://www.foolabs.com/xpdf/home.html>

The GetText utility is from\ <http://www.kryltech.com> with a license in the file GetText.txt (in the EdSharp\Convert\GetText folder).

I welcome feedback, which helps EdSharp improve over time. When reporting a problem, the more specifics the better, including steps to reproduce it, if possible.

The latest version of EdSharp is available at the same URL,\

http://www.EmpowermentZone.com/EdSharp_setup.exe This may be downloaded and installed with the Elevate Version command, F11.

Jamal Mazrui jamal at empowermentzone dot com